



同濟大學
TONGJI UNIVERSITY

嵌入式系统课程实验课题报告

——基于 STM32H747I-DISCO 开发板的 人脸检测系统实验

姓 名	:	杨玺威	王伯铭	袁野	申逸秋
学 号	:	2453360	2453992	2452758	2453598
学 院	:	电子与信息工程学院			
专 业	:	人工智能	人工智能	人工智能	自动化
指导教师	:	徐和根			

2026 年 6 月 12 日

1 实验目的

本实验旨在基于 STM32H747I-DISCO 双核开发板，综合运用 STM32CubeMX、Keil MDK 与 HAL 库，掌握嵌入式 AI 系统的完整开发流程，实现从图像输入、AI 推理到结果可视化的闭环应用。

硬件层面：重点学习 DSI 命令模式、LTDC 与 DMA2D 协同驱动的 LCD 显示机制，掌握 SDRAM 帧缓冲区的内存分配与数据搬运策略；同时通过 USART 中断接收 PC 端图像数据，构建采集、缓存、预处理与显示交替运行的稳定数据流。

AI 部署层面：熟悉 STM32Cube AI Studio 工具链，完成 YuNet 模型从 ONNX 格式到嵌入式 C 代码的转换与集成，针对 NCHW 张量布局、BGR 通道顺序和量化精度等关键要素进行适配验证，确保推理正确性。

后处理与系统集成：自主实现检测框解码、非极大值抑制及边界框绘制，强化对深度学习数学逻辑与内存一致性的理解。最终成功搭建“PC 发送图像-串口接收-AI 推理-LCD 显示检测框”的完整系统，并通过反复调试积累软硬件协同排查经验，为后续边缘智能系统开发奠定技术基础。

2 实验分工

姓名	分工
杨玺威	CubeMX 工程配置和 AI 模型部署
王伯铭	Python 程序和串口图片输入
袁野	AI Studio 模型配置和开发板调试
申逸秋	LCD 显示和图片预处理

3 实验平台

3.1 硬件平台

项目	规格
开发板	STM32H747I-DISCO
MCU	STM32H747XI (双核: Cortex-M7 @ 480 MHz + Cortex-M4 @ 240 MHz)
显示面板	OTM8009A (800×480 RGB888, MIPI DSI Command Mode)

SDRAM	IS42S32800J, 32MB @ 0xD0000000
串口	USART1 (PA9-TX, PA10-RX), 115200 baud
连接	ST-Link/V3 板载调试器

3.2 软件平台

工具	版本/说明
配置工具	STM32CubeMX, 6.17.0 version
IDE	Keil MDK-ARM uVision 5, ARMCC V5.06 update 6
设备包	Keil.STM32H7xx_DFP.2.6.0
AI 工具链	STM32Cube AI Studio
PC 端脚本	Python 3 + pyserial + Pillow
模型	YuNet-320
其他工具	STM32CubeProgrammer, 2.22.0 version



Figure 2. B-LCD40-DSI1 daughterboard hardware block diagram

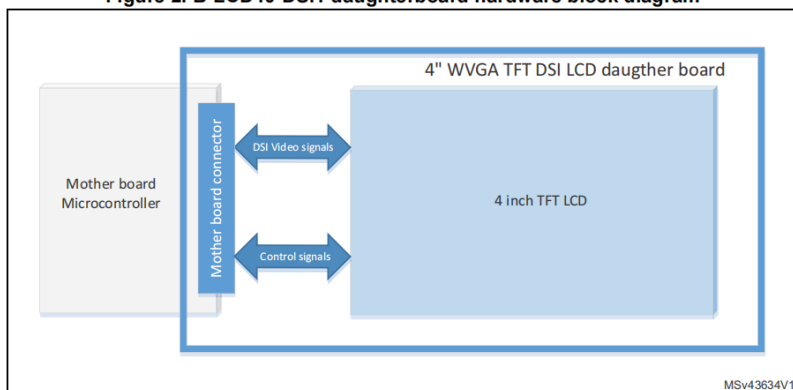
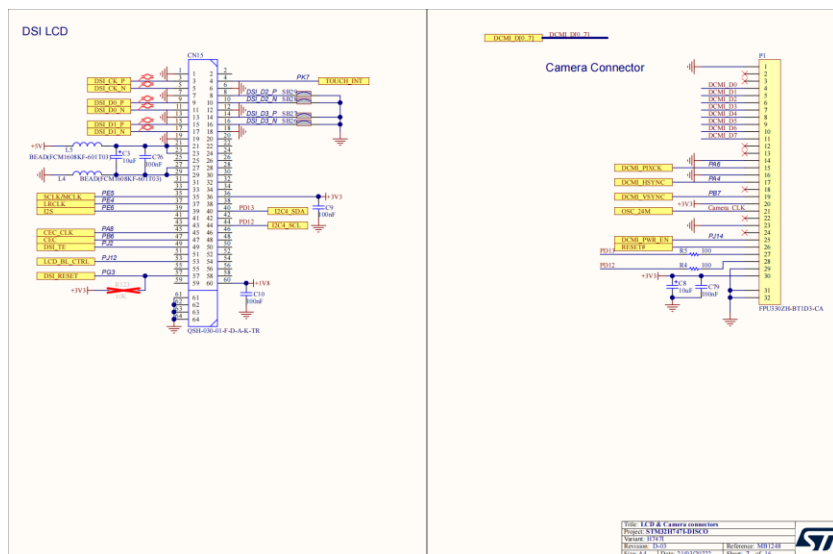
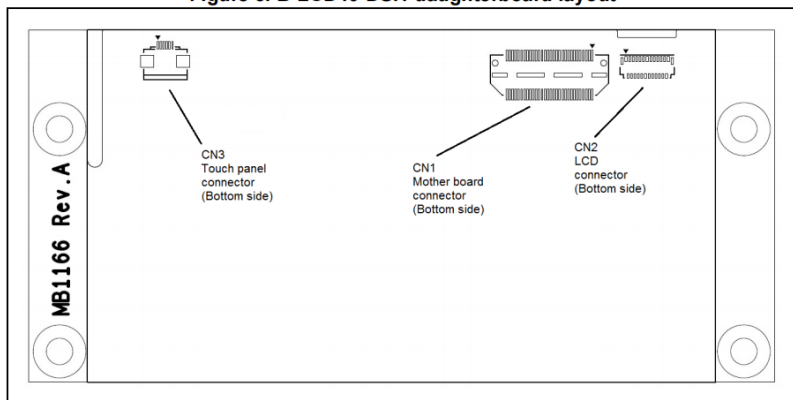
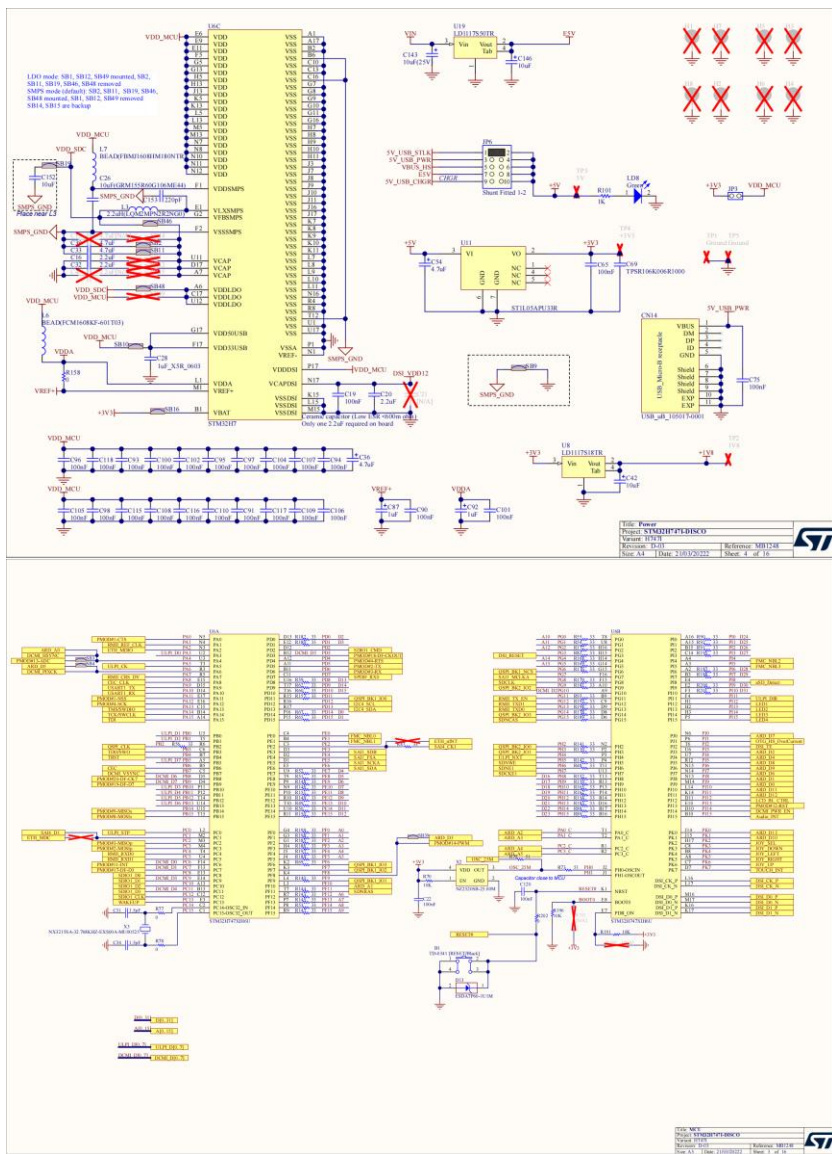


Figure 3. B-LCD40-DSI1 daughterboard layout

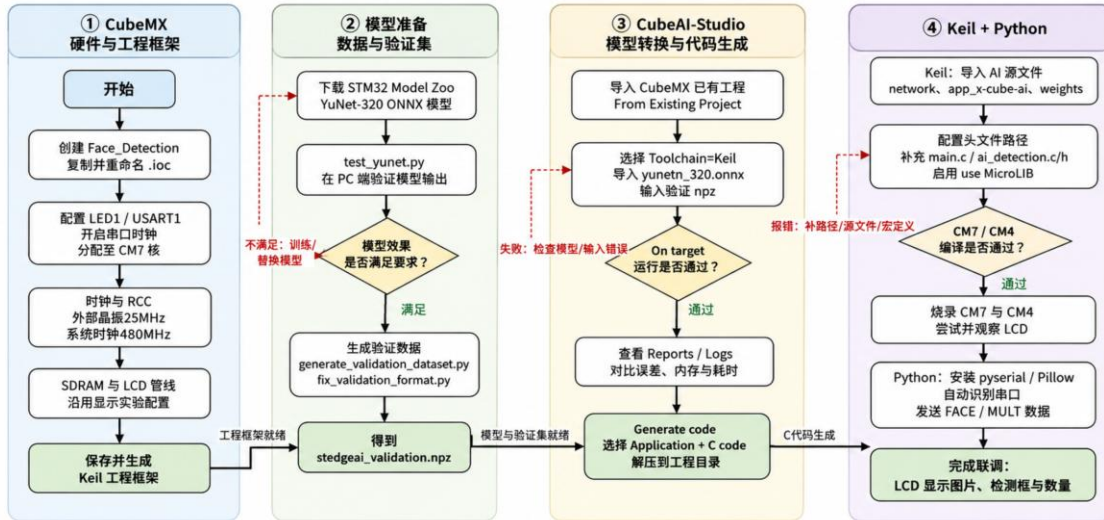




4 实验步骤

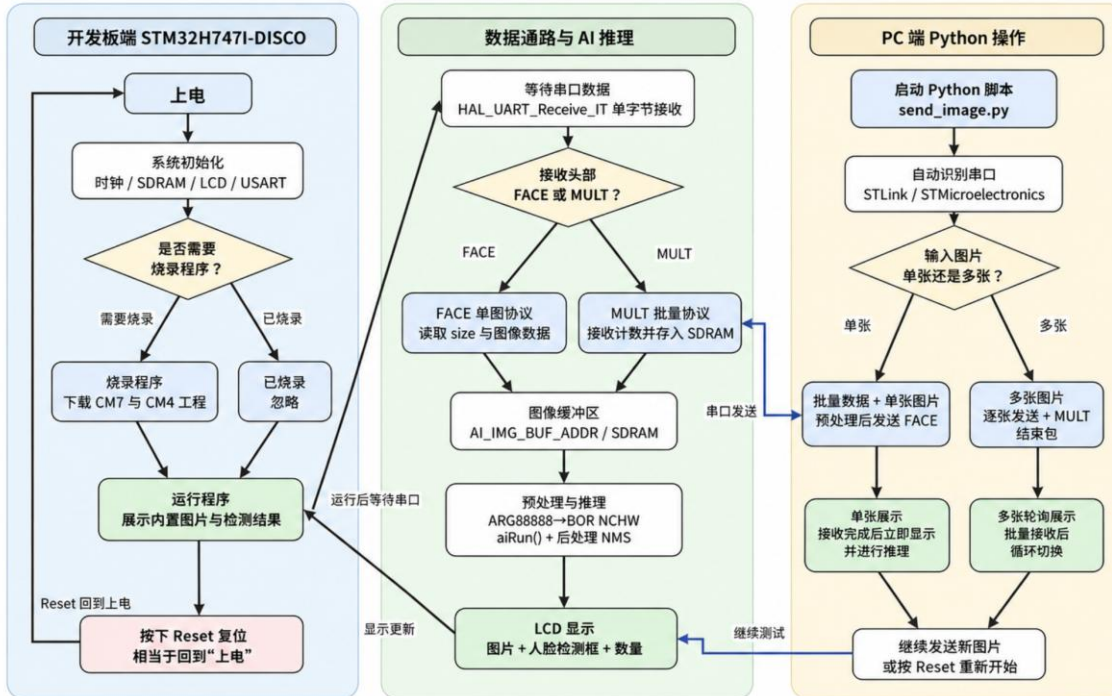
工程构建与部署流程图

从 CubeMX 工程创建到 CubeAI 模型生成、Keil 集成及 Python 串口测试



开发板使用流程图

包含上电初始化、烧录/已烧录分支、串口输入、单图/多图展示与 Reset 回到上电

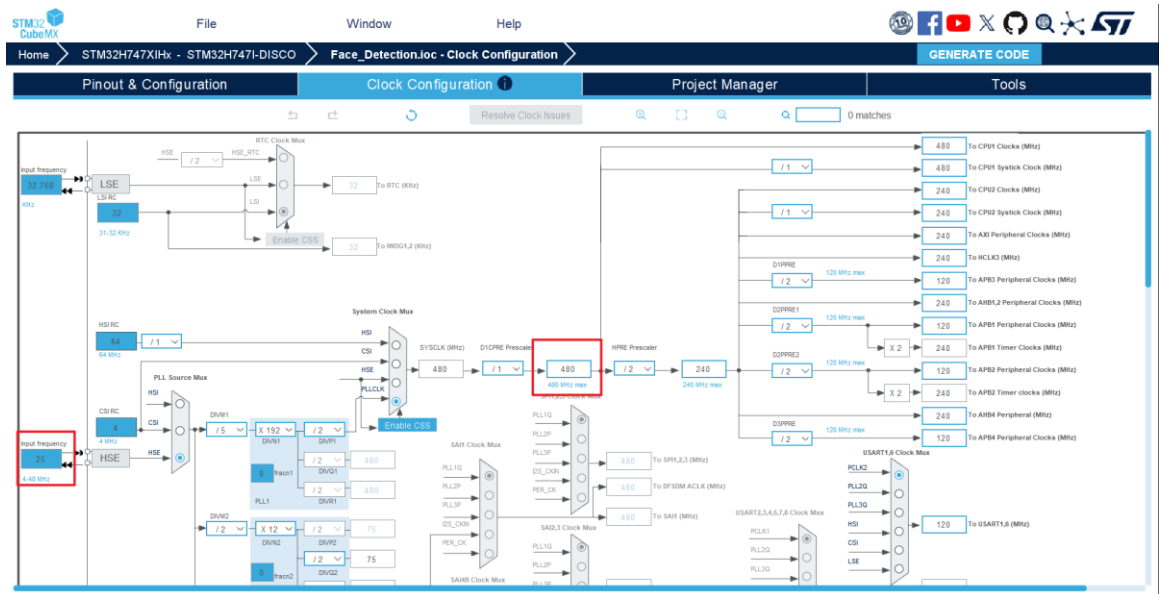


4.1 STM32CubeMX 初始化配置

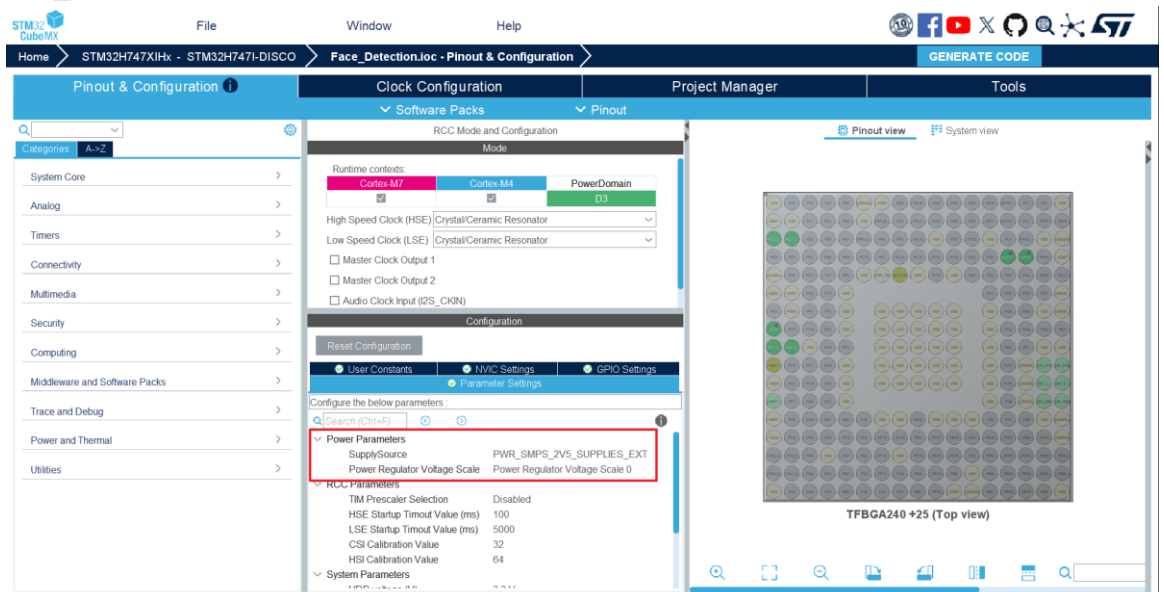
1) 创建新文件夹 Face_Detection，并将 LCD_DSI_CMD_mode_Single_Buffer 实验项目中的 LCD_DSI_CMD_mode_Single_Buffer.ico 文件拷贝到该文件夹，然后重命名为 Face_Detection.ico。

个 UART1 口，开启串口中断，其他外设可忽略。上述外设均分配到 CM7 核。

并回车确定，提高时钟频率以提高 CPU 效率。



Power Regulator Voltage Scale 0, 修改 Supply Source 为 PWR SMPS 2V5 SUPPL



5) SDRAM 内存配置同 LCD_DSI_CMD_mode_Single_Buffer 工程即可。

6) 其它外设保持默认，保存并生成工程代码框架。

4.2 人脸识别模型准备

1) 从 Github 上 STM32 官方模型动物园 ([STM32 Model Zoo](#)) 下载预训练好的模型。本实验选用了 yunetn_320.onnx 模型。

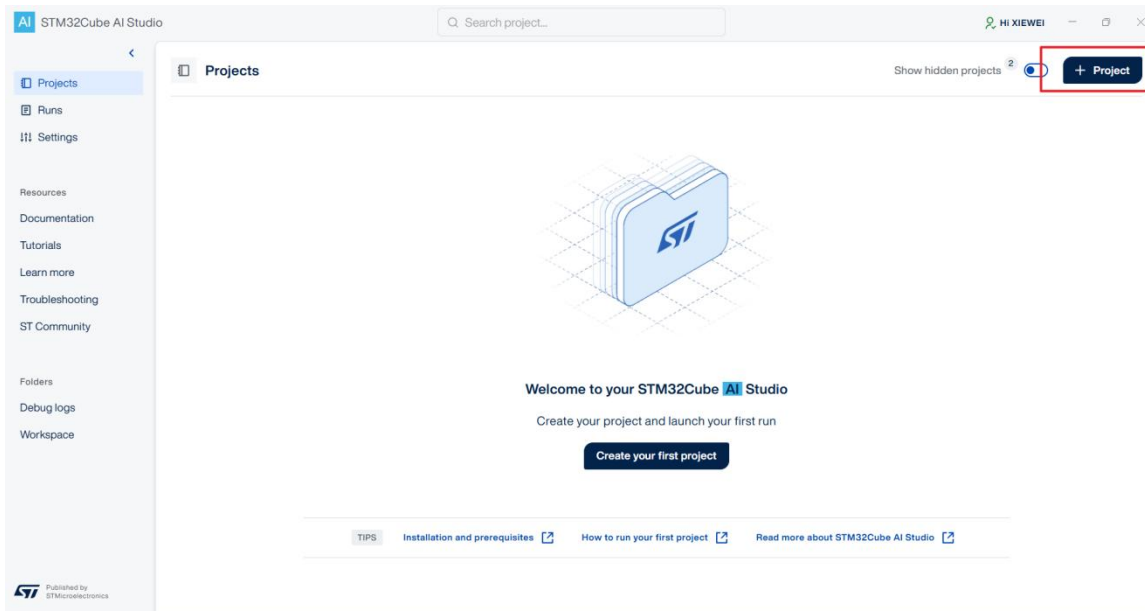
2) 编写 python 程序 (test_yunet.py) 测试模型, 如果达到预期要求, 则直接使用这个模型; 如果没有或者需要使用自己的数据集, 可以对这个模型进行训练。本实验直接使用了这个模型。

3) 本次实验使用了公开数据集 WIDER FACE^①。

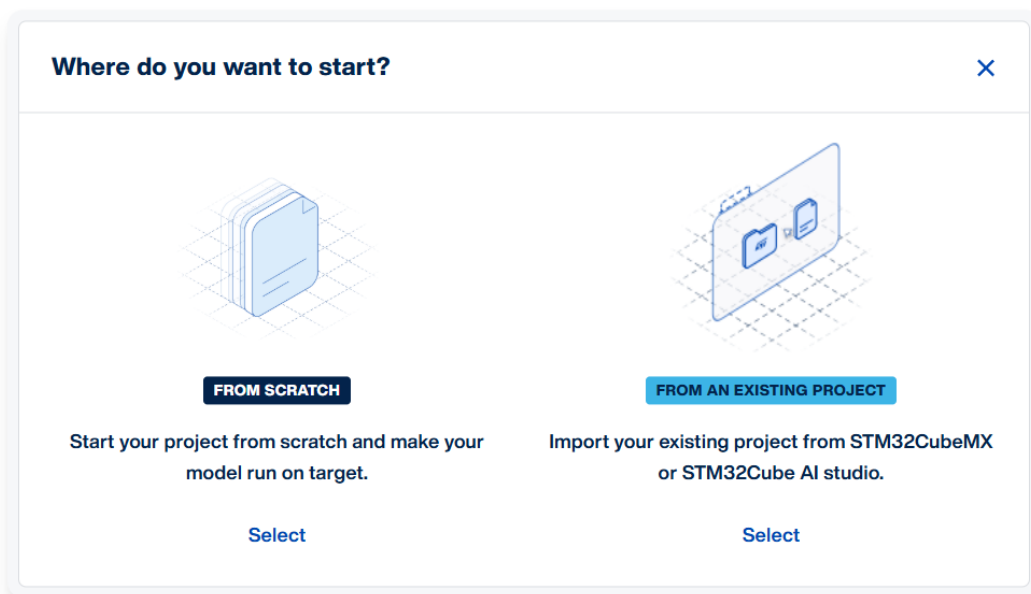
4) 编写 python 程序 (generate_validation_dataset.py 和 fix_validation_format.py), 生成 stedgeai_validation.npz 文件, 用于在 AI-Studio 输入数据测试、验证模型。

4.3 STM32Cube-AI Studio 配置模型文件

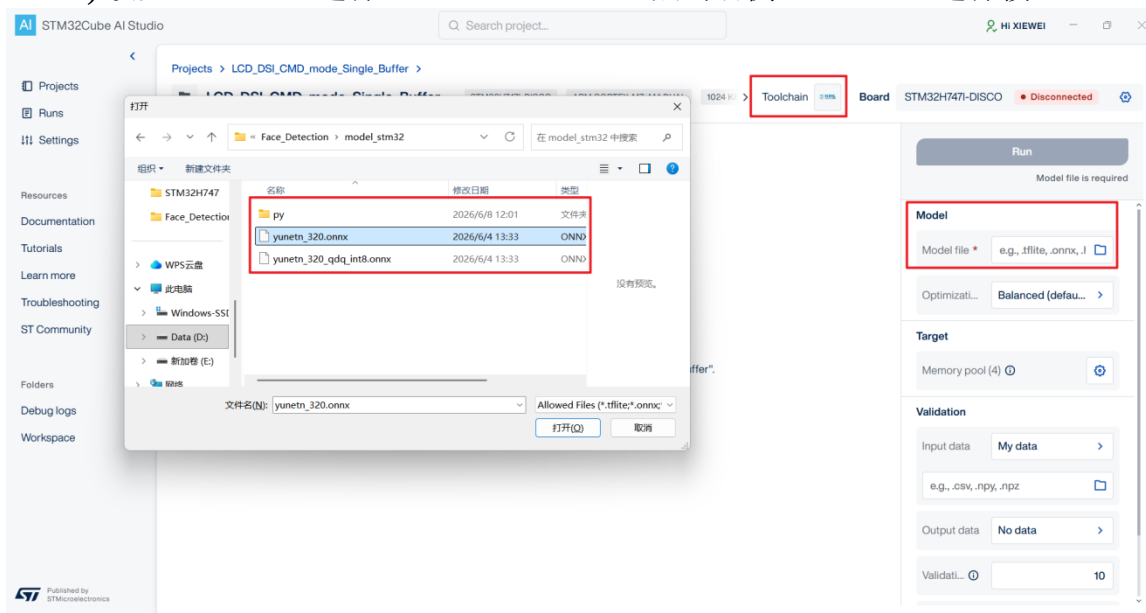
1) 双击打开, 点击右上角 “+Project”, 选择右侧 FROM AN EXISTING PROJECT, 找到 CubeMX 生成的工程文件, 选择对应文件夹, 此选项无需选择开发板型号, 能够自动读取 CubeMX 的配置。



① 数据集官网: <http://shuoyang1213.me/WIDERFACE/>; 数据集规模: 32203 张图像, 标注人脸总计 393703 个; 按规范划分为训练集、验证集、测试集。



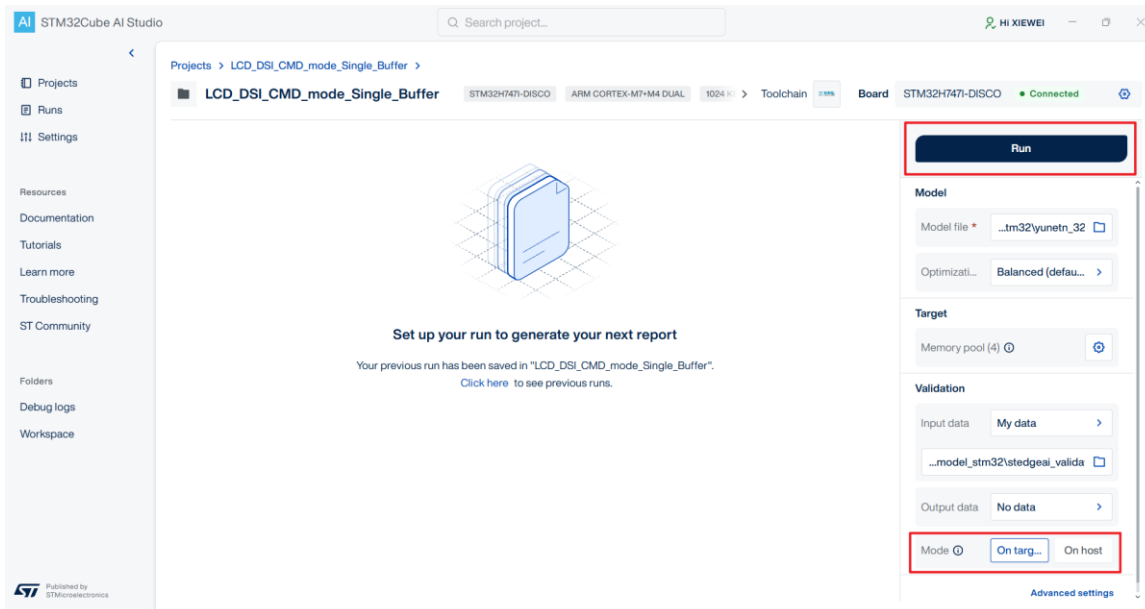
2) 设置 Toolchain 选择 Keil MDK-ARM, 点击右侧 Model file 选择模型。



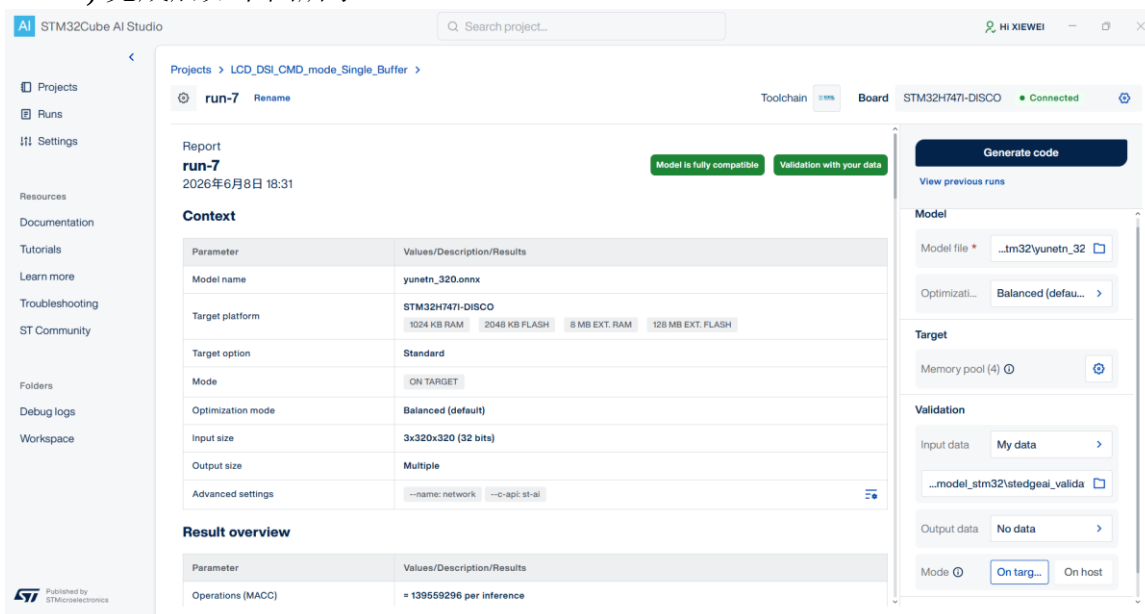
3) 点击 input data 选择 My data, 在下方文件路径选择 4.2 节生成的数据文件 s tedgeai_validation.npz, 注意事项见 5.1 节。

4) 随后可选择 Mode, On target 即在开发板上运行, On host 即在电脑上运行。

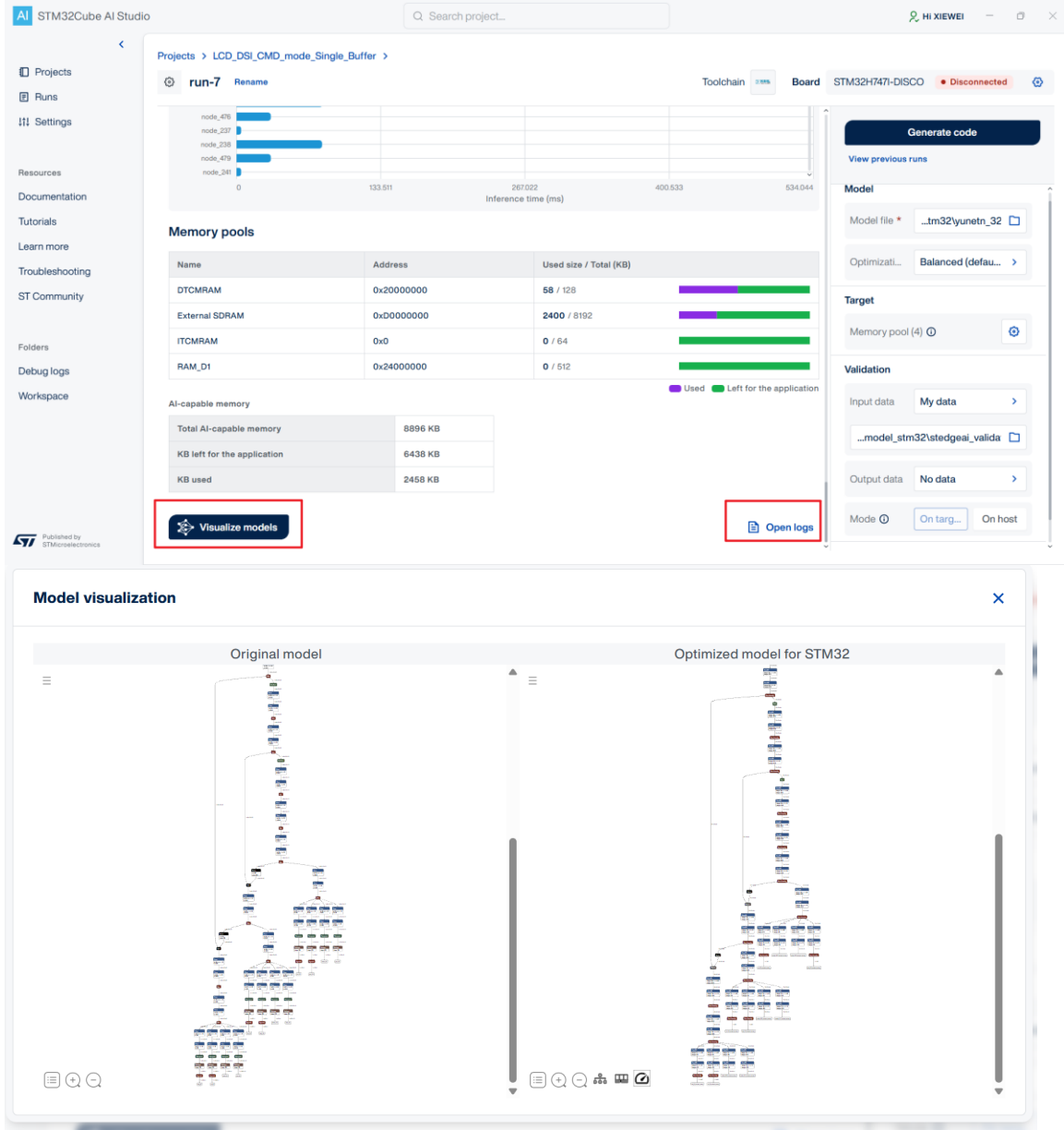
5) 确定 Mode 后, 点击 run, AI Studio 会自动调用 CubeMX, CubeProgrammer, 等待运行完成。



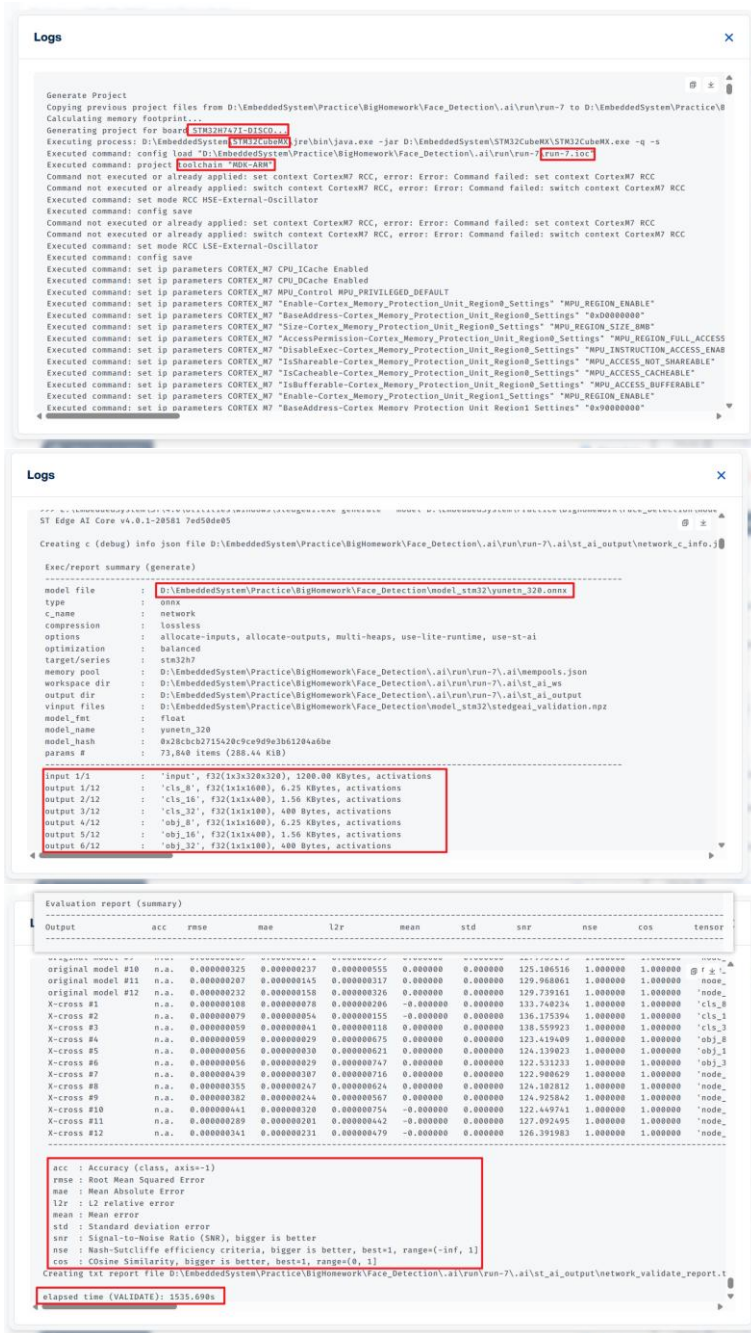
6) 完成后如下图所示



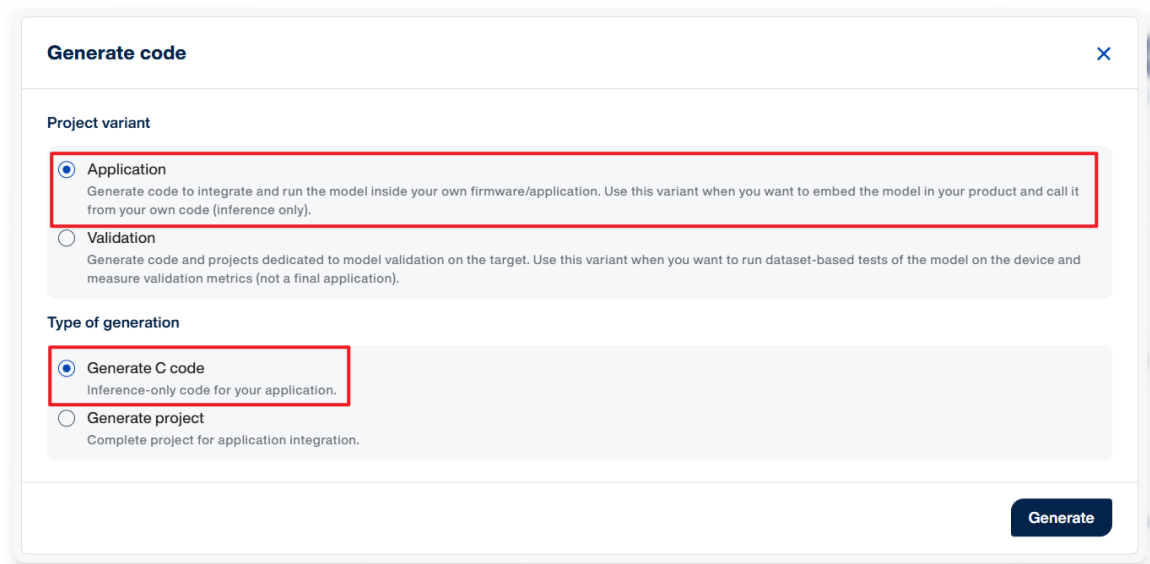
7) 划到页面最低端，选择 Visualize models，可以可视化对比原始模型和为 ST M32 优化的 C 语言模型。



8) 选择 Open logs，查看 AI Studio 运行过程，第一张图展示了开发板型号、调用 CubeMx 的过程、使用 MDK-ARM 即 Keil5 编译运行等；第二张图展示了模型基本信息；第三张图展示了该模型推理所得输出与原模型输出的各类误差，以及运行总时间。中间有大量编译、烧录信息，以及模型推理、神经网络节点数据等内容，由于篇幅有限，不在此处列出。



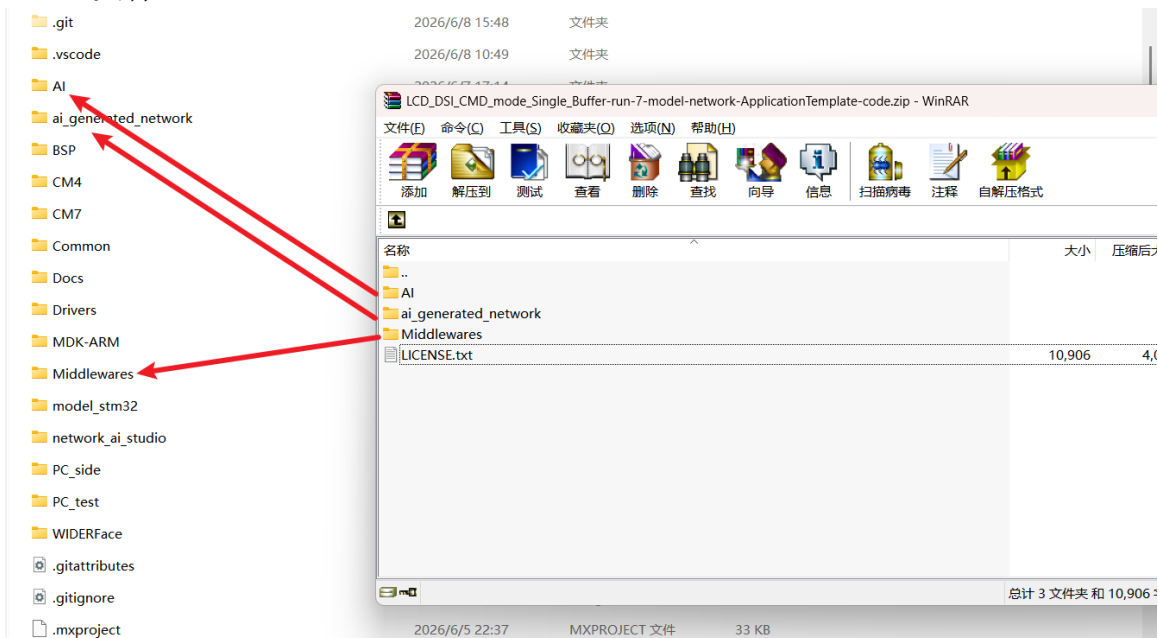
9) 点击 Generate code, 生成 C 语言模型代码, 选择 Application 和 Generate Cc ode, 注意事项见 5.1 节。把生成的压缩包解压到 Face_Detection 文件夹中。



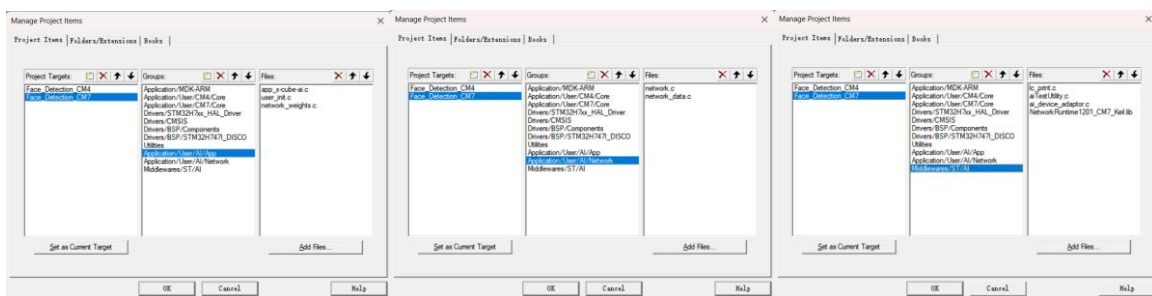
4.4 Keil 环境下编程

1) 恢复此前基于 BSP 的 LCD 显示实验中 Keil 的环境配置，包括导入 C 语言文件和头文件搜索路径，可先编译、烧录保证屏幕能够正常显示。

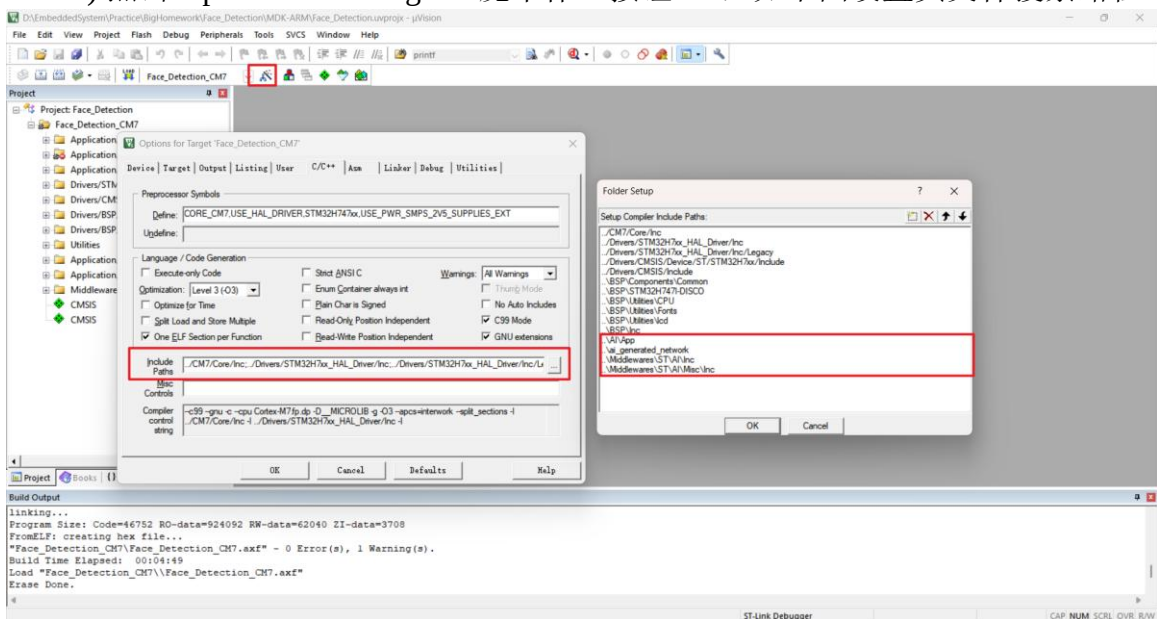
2) 检查由 AI Studio 生成的文件是否都正确放入工程目录中，注意保留 LICENSE.txt 文件。



3) 点击 File Extensions, Books and Environment (立方体) 按钮 ，如下图加入 AI 模型相关的 C 语言文件。



4) 点击 Options For Target（魔术棒）按钮 ，如下图设置头文件搜索路径。



5) 在 CM7 的 main.c 文件中添加代码，此处只呈现模型相关和串口输入图形相关代码(VSCode 风格)。

```
/* USER CODE BEGIN Includes */
#include "stm32h747i_discovery.h"
#include "stm32h747i_discovery_lcd.h"
#include "stm32h747i_discovery_sdram.h"
#include "stm32h747i_discovery_bus.h"
#include "stm32_lcd.h"
#include "face_img_0.h"
#include "face_img_1.h"
#include <string.h>
#include <stdio.h>
#include <math.h>
#include "app_x-cube-ai.h"
#include "bsp_ai.h"
#include "network.h"
#include "network_data.h"
#include "ai_detection.h"
/* USER CODE END Includes */

/* USER CODE BEGIN PTD */
/* USART RX state machine states */
typedef enum {
    RX_STATE_IDLE = 0, /* Waiting for "FACE" or "MULT" magic header (4 bytes) */
    RX_STATE_GOT_MAGIC, /* Magic received, now reading size/count field (4 bytes) */
    RX_STATE_GOT_SIZE, /* Size parsed, now receiving raw image pixel data */
    RX_STATE_RECEIVING /* (Alias) actively receiving image bytes into SDRAM */
}
```

<pre> } rx_state_t; /* USER CODE END PTD */ /* USER CODE BEGIN PV */ ... /* ---- USART RX state machine variables ---- */ static rx_state_t rx_state = RX_STATE_IDLE; static uint32_t rx_expected_size = 0; static uint32_t rx_received = 0; static uint8_t rx_header_buff[8]; /* 4 magic + 4 size */ static uint8_t rx_header_idx = 0; static uint32_t rx_magic_type = 0; /* FACE or MULT */ static volatile uint8_t rx_img_ready = 0; static uint8_t rx_byte; /* Single-byte RX buffer */ /* ---- Multi-image mode ---- */ static int usart_img_count = 0; /* Number of USART images received */ static int usart_img_index = 0; /* Current display index */ static volatile uint8_t usart_multi_ready = 0; /* MULT end-of-batch received */ /* ---- AI inference variables ---- */ static float *ai_input; /* Model input tensor pointer */ static Detection detections[MAX_DETECTIONS]; /* Face detection results */ static int num_detections = 0; /* ---- SDRAM image buffer (for backward compatibility) ---- */ static uint8_t *usart_img_buf = (uint8_t *)AI_IMG_BUF_ADDR; /* USER CODE END PFP */ </pre>
<pre> /* USER CODE BEGIN 0 */ /* External reference to AI network context (defined in app_x-cube-ai.c) */ extern uint8_t network_context[STAI_NETWORK_CONTEXT_SIZE]; extern stai_ptr stai_input[STAI_NETWORK_IN_NUM]; extern stai_ptr stai_output[STAI_NETWORK_OUT_NUM]; /* ---- USART helper functions ---- */ /* Start single-byte interrupt reception on USART1 */ static void USART_StartRxByte(void) HAL_UART_Receive_IT(&huart1, &rx_byte, 1); /* Get pointer to n-th received image in SDRAM (each image is 307200 bytes) */ static uint32_t *usart_get_img(int n) return (uint32_t *) (AI_IMG_BUF_ADDR + n * USART_RX_IMG_SIZE); /* USER CODE END 0 */ </pre>
<pre> /* USER CODE BEGIN 2 */ ... /* Initialize Cube.AI runtime and model */ STM32CubeAI_Studio_AI_Init(); ai_input = (float *)stai_input[0]; /* Cache input tensor: 1x3x320x320 float32 */ /* Start USART1 RX interrupt for image reception */ USART_StartRxByte(); /* USER CODE END 2 */ </pre>
<pre> /* ---- Check USART-received image ---- */ if (rx_img_ready) { rx_img_ready = 0; use_builtin = 0; sprintf((char *)My_String, "USART: rcv %d image(s)", usart_img_count); UTIL_LCD_DisplayStringAtLine(7, (uint8_t *)My_String); } </pre>
<pre> /* Step 1: Copy image to LCD frame buffer via DMA2D */ CopyBuffer(src_img, (uint32_t *)LCD_FRAME_BUFFER, 240, 160, 320, 240); /* Step 2: Preprocess - ARGB8888 to NCHW BGR float32 */ ai_preprocess(src_img, 320, 240, ai_input, 320, 320); /* Clean D-Cache so ST AI runtime reads correct input from SDRAM */ SCB_CleanDCache_by_Addr((uint32_t *)ai_input, (int32_t)(3 * 320 * 320 * sizeof(float))); /* Step 3: Run AI inference */ stai_return_code ret = aiRun(); if (ret != STAI_SUCCESS) { sprintf((char *)My_String, "AI err: %d", (int)ret); UTIL_LCD_DisplayStringAtLine(26, (uint8_t *)My_String); } else { /* Step 4: Post-process - decode detections + NMS + containment filter */ num_detections = ai_postprocess(stai_output, detections, MAX_DETECTIONS, DET_THRESHOLD, NMS_THRESHOLD, MIN_BOX_SIZE); /* Step 5: Draw green bounding boxes on LCD frame buffer */ </pre>


```

ai_draw_detections(detections, num_detections,
                  (uint32_t *)LCD_FRAME_BUFFER,
                  800, 240, 160, 320, 240);
/* Display detection count below the image (Line 26, Font16, y~416) */
UTIL_LCD_SetBackColor(UTIL_LCD_COLOR_WHITE);
UTIL_LCD_SetTextColor(UTIL_LCD_COLOR_BLACK);
sprintf((char *)My_String, "Faces detected: %d", num_detections);
UTIL_LCD_DisplayStringAtLine(26, (uint8_t *)My_String);
}

```

6) 添加 ai_detection.c 和 ai_detection.h 文件，导入工程及头文件路径；ai_detection.c 主要代码如下。

```

#include "ai_detection.h"
#include <math.h>
/* YuNet anchor configuration for 320x320 input (stride only) */
const int yu_strides[3] = YU_STRIDES;
const int yu_grid_sizes[3] = YU_GRID_SIZES;

void ai_preprocess(uint32_t *src, int src_w, int src_h, float *dst, int dst_w, int dst_h) {
    int c, y, x, sy, sx;
    uint8_t r, g, b;
    int x_ratio = ((src_w << 16) / dst_w) + 1;
    int y_ratio = ((src_h << 16) / dst_h) + 1;
    int plane_size = dst_h * dst_w; /* Size of one channel plane */
    for (c = 0; c < 3; c++) {
        float *dst_c = dst + c * plane_size; /* NCHW: point to channel c start */
        for (y = 0; y < dst_h; y++) {
            sy = (y * y_ratio) >> 16;
            if (sy >= src_h)
                sy = src_h - 1;
            for (x = 0; x < dst_w; x++) {
                sx = (x * x_ratio) >> 16;
                if (sx >= src_w)
                    sx = src_w - 1;
                uint32_t pixel = src[sy * src_w + sx];
                b = pixel & 0xFF;
                g = (pixel >> 8) & 0xFF;
                r = (pixel >> 16) & 0xFF;
                /* BGR channel order, values in [0, 255] - no normalization */
                float val;
                if (c == 0)
                    val = (float)b;
                else if (c == 1)
                    val = (float)g;
                else
                    val = (float)r;
                dst_c[y * dst_w + x] = val; /* NCHW: channel-first, planar BGR */
            }
        }
    }
}

int ai_postprocess(stai_ptr *outputs, Detection *dets, int max_dets, float threshold, float nms_threshold, float min_box_size)
{
    int det_count = 0;
    int k, i, j, loc;
    int gs;
    float stride;
    float *cls, *obj, *bbox;
    for (k = 0; k < NUM_STRIDES; k++) {
        stride = (float)yu_strides[k];
        gs = yu_grid_sizes[k];
        cls = (float *)outputs[k];
        obj = (float *)outputs[k + 3];
        bbox = (float *)outputs[k + 6];
        for (i = 0; i < gs; i++) {
            for (j = 0; j < gs; j++) {
                loc = i * gs + j;
                float score = cls[loc] * obj[loc];
            }
        }
    }
}

```

```

        if (score < threshold) continue;
        // bbox in interleaved (N×4) format, not CHW
        float dx = bbox[loc * 4 + 0];
        float dy = bbox[loc * 4 + 1];
        float dw = bbox[loc * 4 + 2];
        float dh = bbox[loc * 4 + 3];
        float cx = ((float)j + dx) * stride;
        float cy = ((float)i + dy) * stride;
        float bw = expf(dw) * stride;
        float bh = expf(dh) * stride;
        if (bw < min_box_size || bh < min_box_size) continue;
        float x1 = cx - bw * 0.5f;
        float y1 = cy - bh * 0.5f;
        float x2 = cx + bw * 0.5f;
        float y2 = cy + bh * 0.5f;
        // clip to image boundary (320×320)
        if (x1 < 0.0f) x1 = 0.0f;
        if (y1 < 0.0f) y1 = 0.0f;
        if (x2 > 320.0f) x2 = 320.0f;
        if (y2 > 320.0f) y2 = 320.0f;
        if (x2 <= x1 || y2 <= y1) continue;
        if (det_count < max_dets) {
            dets[det_count].x1 = x1;
            dets[det_count].y1 = y1;
            dets[det_count].x2 = x2;
            dets[det_count].y2 = y2;
            dets[det_count].score = score;
            det_count++;
        }
    }
}

// Sort by descending score (bubble sort for brevity)
for (int a = 0; a < det_count - 1; a++)
    for (int b = a + 1; b < det_count; b++)
        if (dets[b].score > dets[a].score) {
            Detection t = dets[a];
            dets[a] = dets[b];
            dets[b] = t;
        }

// NMS
for (int a = 0; a < det_count; a++) {
    if (dets[a].score <= 0.0f) continue;
    float area_a = (dets[a].x2 - dets[a].x1) * (dets[a].y2 - dets[a].y1);
    for (int b = a + 1; b < det_count; b++) {
        if (dets[b].score <= 0.0f) continue;
        float xx1 = fmaxf(dets[a].x1, dets[b].x1);
        float yy1 = fmaxf(dets[a].y1, dets[b].y1);
        float xx2 = fminf(dets[a].x2, dets[b].x2);
        float yy2 = fminf(dets[a].y2, dets[b].y2);
        float iw = xx2 - xx1, ih = yy2 - yy1;
        if (iw <= 0.0f || ih <= 0.0f) continue;
        float inter = iw * ih;
        float area_b = (dets[b].x2 - dets[b].x1) * (dets[b].y2 - dets[b].y1);
        float iou = inter / (area_a + area_b - inter);
        float smaller_area = (area_a < area_b) ? area_a : area_b;
        if (iou > nms_threshold || inter > 0.75f * smaller_area)
            dets[b].score = -1.0f;
    }
}

// Compact list
int keep = 0;
for (int a = 0; a < det_count; a++)
    if (dets[a].score > 0.0f)
        dets[keep++] = dets[a];
return keep;
}

void ai_draw_detections(Detection *dets, int ndet, uint32_t *fb, int fb_w, int off_x, int off_y, int img_w, int i
mg_h) {
    int d, t, x, y;

```

```

float scale_x = (float)img_w / 320.0f; /* Model input was 320x320 */
float scale_y = (float)img_h / 320.0f;
for (d = 0; d < ndet; d++) {
    int x1 = (int)(dets[d].x1 * scale_x) + off_x;
    int y1 = (int)(dets[d].y1 * scale_y) + off_y;
    int x2 = (int)(dets[d].x2 * scale_x) + off_x;
    int y2 = (int)(dets[d].y2 * scale_y) + off_y;
    if (x1 < 0) x1 = 0;
    if (y1 < 0) y1 = 0;
    if (x2 >= fb_w) x2 = fb_w - 1;
    if (y2 >= 480) y2 = 479;
    uint32_t green = 0xFF00FF00; /* ARGB8888 Green */
    for (t = 0; t < 2; t++) {
        int bx1 = x1 + t, by1 = y1 + t;
        int bx2 = x2 - t, by2 = y2 - t;
        if (bx2 <= bx1 || by2 <= by1)
            break;
        /* Top & bottom edges */
        for (x = bx1; x <= bx2; x++) {
            fb[by1 * fb_w + x] = green;
            fb[by2 * fb_w + x] = green;
        }
        /* Left & right edges */
        for (y = by1; y <= by2; y++) {
            fb[y * fb_w + bx1] = green;
            fb[y * fb_w + bx2] = green;
        }
    }
}
}

```

7) 完成代码编写后，点击魔术棒图标设置工程选项，选中启用 use MicroLIB。

8) 保存、编译，确保没有提示出错信息，同样切换到 CM4，编译，确保没有提示出错信息。

9) 连接开发板，下载 CM7 和 CM4 程序，按下复位键开始运行；等待片刻，观察 LCD 显示屏文字和图片显示是否符合预期。

10) 具体屏幕显示结果见附件 1: 人脸识别演示视频/Face Detection Demo Video; 踩坑记录与注意事项见 5.3 节。

4.5 基于 Python 的串口图片输入

1) 首先在 python 环境中安装必要的库 pyserial 和 pillow。

2) 编写 python 代码，加入串口自动识别和连接、单/多张图片输入、图片预处理相关代码。

```

def find_serial_port():
    """Auto-detect STM32 serial port."""
    import serial.tools.list_ports
    ports = list(serial.tools.list_ports.comports())
    for p in ports:
        if "STMMicroelectronics" in p.description or "STLink" in p.description:
            print(f"Found STM32 port: {p.device} - {p.description}")
            return p.device
    if ports:
        print(f"No STM32 port found. Available: {[p.device for p in ports]}")
        return ports[0].device
    print("No serial ports found!")
    return None
def send_single_image(ser, raw_data, img_index, total):

```

```

"""Send one image with FACE protocol. Returns est. time in seconds."""
total_size = len(raw_data)
header = MAGIC_FACE + struct.pack("<I", total_size)
payload = header + raw_data
print("\n Image {img_index}/{total}: {total_size} bytes ->", end=" ", flush=True)
CHUNK = 4096
sent = 0
while sent < len(payload):
    chunk = payload[sent:sent + CHUNK]
    ser.write(chunk)
    sent += len(chunk)
print("OK")
return len(payload) * 10.0 / BAUDRATE
def send_mult_end(ser, num_images):
    """Send MULT end-of-batch packet. STM32 enters multi-image cycle mode."""
    header = MAGIC_MULT + struct.pack("<I", num_images) + struct.pack("<I", 0)
    ser.write(header)
    print(f"\n Sent MULT: {num_images} image(s) total. STM32 will cycle through them.")

```

3) 打开终端或者命令行界面使用如下格式输入图片测试。

```

# 单张图片（开发板自动识别并循环显示检测结果）
python PC_side/send_image.py path/to/face.jpg
# 整个文件夹（最大 10 张，开发板循环显示）
python PC_side/send_image.py WIDERFace/WIDER_test/images/0--Parade

```

4) 单张图片结果。

```

C:\WINDOWS\system32\cmd: x + v
C:\Users\14821>D:\PythonConda\envs\pytorch_face_detection\python.exe D:\EmbeddedSystem\Practice\BigHomework\Face_Detection\PC_side\send_image.py D:\EmbeddedSystem\Practice\BigHomework\Face_Detection\PC_side\test_photo\1_Handshaking_Handshaking_1_827.jpg
Found STM32 port: COM4 - STMicroelectronics STLink Virtual COM Port (COM4)

=====
Sending 1 image(s) to COM4 at 115200 baud
=====
Opened COM4

[1/1] D:\EmbeddedSystem\Practice\BigHomework\Face_Detection\PC_side\test_photo\1_Handshaking_Handshaking_1_827.jpg
Original: 1024x684

Image 1/1: 307200 bytes -> OK

Serial port closed.

Finished! Sent 1 image(s).

```

5) 多张图片结果。

```

C:\WINDOWS\system32\cmd: x + v
C:\Users\14821>D:\PythonConda\envs\pytorch_face_detection\python.exe D:\EmbeddedSystem\Practice\BigHomework\Face_Detection\PC_side\send_image.py D:\EmbeddedSystem\Practice\BigHomework\Face_Detection\PC_side\test_photo
Found 5 image(s) in folder.
Found STM32 port: COM4 - STMicroelectronics STLink Virtual COM Port (COM4)

=====
Sending 5 image(s) to COM4 at 115200 baud
=====
Opened COM4

[1/5] D:\EmbeddedSystem\Practice\BigHomework\Face_Detection\PC_side\test_photo\11_Meeting_Meeting_11_401.jpg
Original: 1024x683

Image 1/5: 307200 bytes -> OK

[2/5] D:\EmbeddedSystem\Practice\BigHomework\Face_Detection\PC_side\test_photo\1_Handshaking_Handshaking_1_827.jpg
Original: 1024x684

Image 2/5: 307200 bytes -> OK

[3/5] D:\EmbeddedSystem\Practice\BigHomework\Face_Detection\PC_side\test_photo\33_Running_Running_33_119.jpg
Original: 1024x768

Image 3/5: 307200 bytes -> OK

[4/5] D:\EmbeddedSystem\Practice\BigHomework\Face_Detection\PC_side\test_photo\4_Dancing_Dancing_4_854.jpg
Original: 1024x768

Image 4/5: 307200 bytes -> OK

[5/5] D:\EmbeddedSystem\Practice\BigHomework\Face_Detection\PC_side\test_photo\54_Rescue_firemanrescue_54_660.jpg
Original: 1024x1531

Image 5/5: 307200 bytes -> OK

Sent MULT: 5 image(s) total. STM32 will cycle through them.

Serial port closed.

Finished! Sent 5 image(s).

```

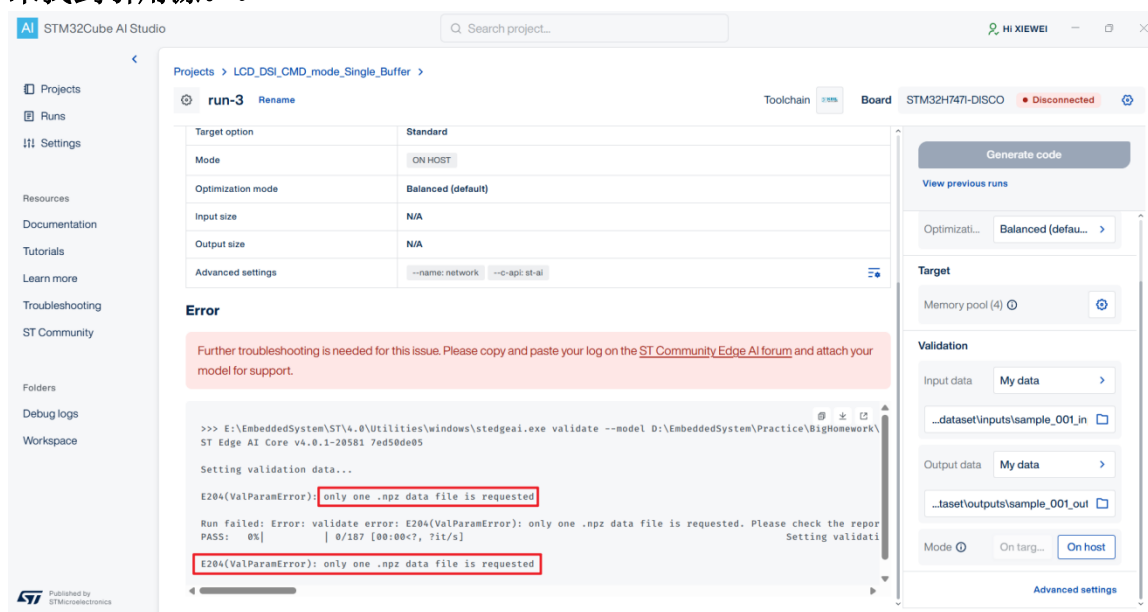
6) 具体屏幕显示结果见附件 1：人脸识别演示视频/Face Detection Demo Video。

5 踩坑记录与注意事项

5.1 STM32Cube-AI Studio 偶现 bug 和数据导入注意事项

1) 在 Cube-AIStudio 中, 选择 FROM AN EXISTING PROJECT 选项打开 Face _Detection 文件夹, 但是项目名称会被识别为 LCD_DSI_CMD_mode_Single_Buffer, 此 bug 偶尔出现且无法更改该工程文件名字。

2) 对于输入数据, 建议使用 python 包 numpy 的文件.npz, 只需要输入 input data 即可, 不需要 Output data, 否则会报错; 具体输入详细参考下图和参考资料错误! 未找到引用源。。



The screenshot shows the STM32Cube AI Studio interface. The 'run-3' project is selected. The 'Error' section displays the following message:

```

E204(ValParamError): only one .npz data file is requested
Run failed: Error: validate error: E204(ValParamError): only one .npz data file is requested. Please check the report
PASS: 0% | 0/187 [00:00<7, 7it/s]
E204(ValParamError): only one .npz data file is requested
  
```

The bottom part of the image shows a table with the following content:

File Type	Description
pb	function. 简单的二进制张量 Proto 文件, 带有 单阵列。(批次大小, -1) 形状。可以用 <code>tf.make_tensor_proto</code> 帮手功能。
npz	Binary numpy file with several arrays. 带有多个数组的二进制数字文件。

For the **npz** file, the following pairs of keys (dict entries) are supported (see the snippet code in FAQ article to generate a .npz file from an image dataset):

对于 **npz** 文件, 支持以下键对 (dict 条目) (请参见常见问题文章中的片段代码, 从图像数据集生成 .npz 文件):

- `x_test` and `y_test` (simple I/O)
`x_test` 和 `y_test` (简单 I/O)
- `inputs` and `outputs` (simple I/O)
输入 和 输出 (简单 I/O)
- `in_0` and `out_0` (simple I/O)
`in_0` 和 `out_0` (简单 I/O)
- `m_inputs` and `m_outputs` (simple I/O)
`m_inputs` 和 `m_outputs` (简单 I/O)
- `m_inputs_idx` and `m_outputs_idx` (multiple I/O, `idx` starting with 1)
`m_inputs_idx` 和 `m_outputs_idx` (多输入输出, `idx` 从 1)
- `c_inputs_idx` and `c_outputs_idx` (multiple I/O, if no `m_inputs_idx` keys are defined)
`c_inputs_idx` 和 `c_outputs_idx` (如果没有, 则多 I/O `m_inputs_idx` 键已定义)

否则, `xxx` 键被视为多个或简单输入, 因此不考虑输出

For the **csv** files, a particular tag (`'dtype=uint8'` or `'dtype=int8'`) is defined inside one of the first five comment lines to indicate the type of the data else float32 data type is used.

对于 **csv** 文件, 在前五条注释行中的某一处定义了特定标签 (`'dtype=uint8'` 或 `'dtype=int8'`), 用以表示数据类型, else float32 数据类型。

3) 在最后一步 Generate code 中，若选择 Generate project，则会导致原项目被覆盖，需要再次配置 CubeMx 和设置 Keil5 的 C 语言文件和头文件路径。

5.2 开发板完全锁死解决方案

1) 由于错误配置时钟参数，导致开发板锁死，尝试使用实验指导书方法解锁，发现 CubeProgrammer 也无法正常连接开发板。

2) 观察开发板指示灯现象：上电后快速闪烁，立即变红灯常亮，猜测开发板被完全锁死。

3) 查阅参考资料**错误!未找到引用源。**，得知解决方案如下，并查得电路板引脚图。

If a deadlock occurs because of a mismatch between hardware and firmware PWR settings (SMP/LDO), the user can recover the board by applying the following procedure:

若因硬件与固件电源配置（开关电源 SMPS / 低压差线性稳压器 LDO）不匹配造成芯片锁死，用户可按照以下步骤恢复开发板：

1. Power off the board.

断开开发板电源

2. Connect pin BOOT0 to VDD using a wire (or short R192).

使用导线将 BOOT0 引脚接至 VDD（或短接电阻 R192）

3. This changes the BOOT0 pin to 1 instead of 0, thus changing the device boot address to boot address 1 and making the bootloader start in System memory. This avoids starting firmware in the user flash memory with a wrong SMP/LDO configuration versus the hardware board configuration.

此举可将 BOOT0 电平由 0 改为 1，设备启动地址切换至地址 1，从系统存储区启动内置引导程序；以此规避从用户闪存启动固件时，出现电源拓扑配置与板载硬件不匹配的问题。

4. Power on the board and connect using STM32CubeProgrammer (STM32CubeProg).

给开发板上电，通过 STM32CubeProgrammer 工具建立连接

5. Erase the user flash memory.

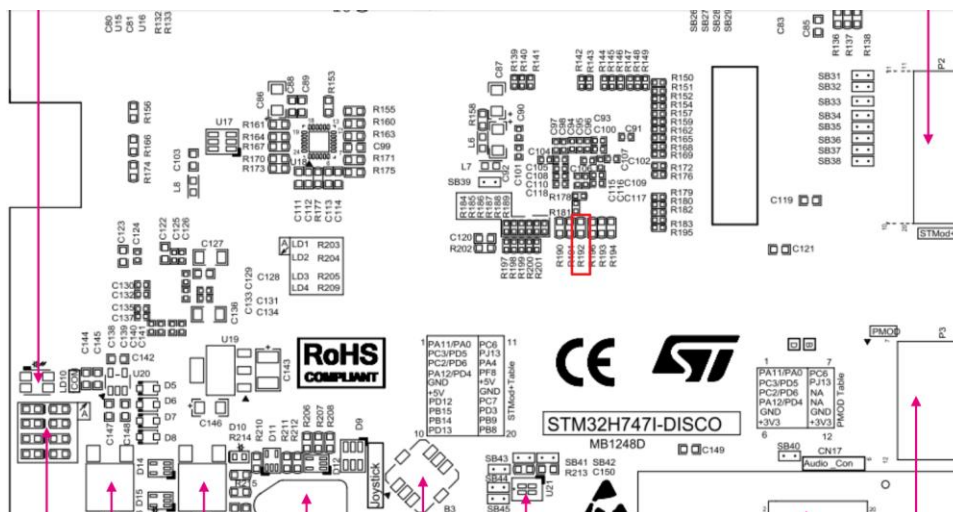
擦除用户闪存区域

6. Power off the board and remove the short between BOOT0 and VDD.

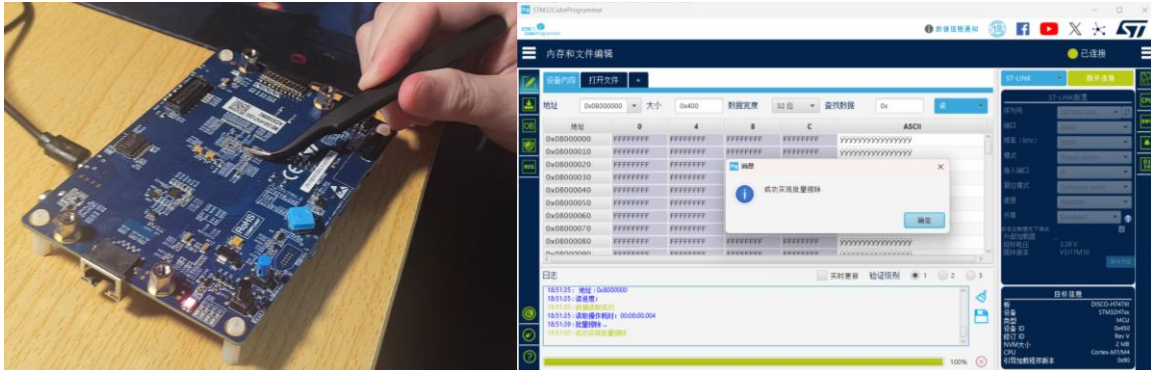
断电，拆除 BOOT0 与 VDD 之间的短接线

7. The board is recovered and can be used normally with matching firmware PWR.

开发板修复完成，烧录电源配置匹配的固件即可正常使用



4) 拆卸屏幕、使用跳线外接或金属镊子短路 R192，再把开发板接入电源，此时使用 CubeProgrammer 可以正常连接，选择完全擦除芯片选项，再断电，去除短接电线。

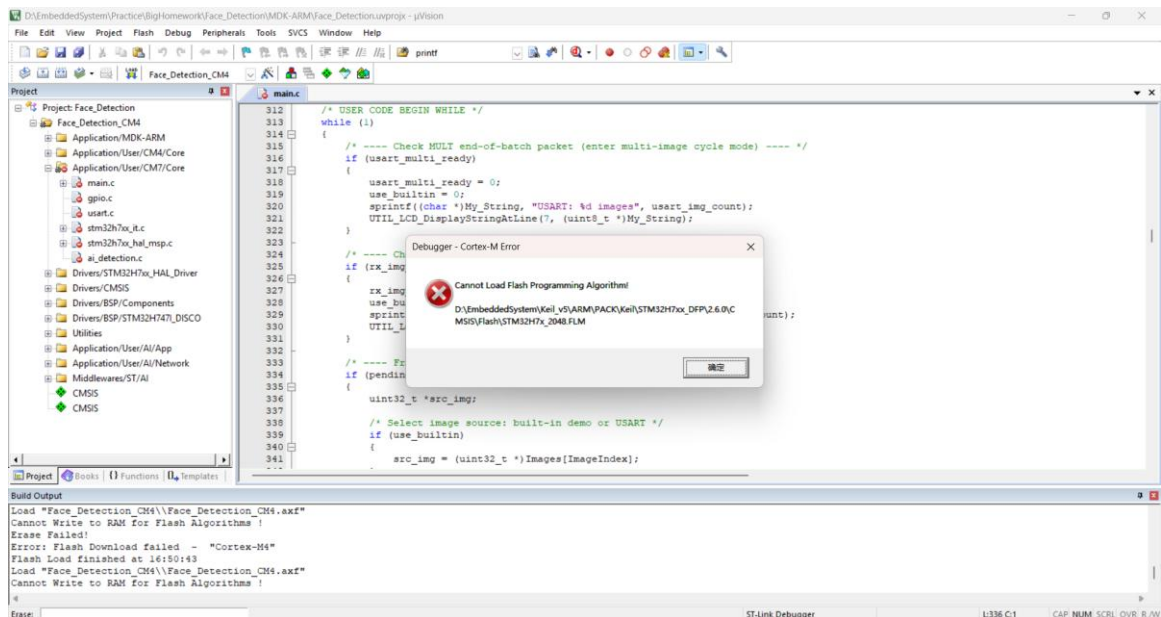


5) 再次上电，烧录正确的 axf 文件测试，开发板正常工作，指示灯正常闪烁。

6) 此修复方法是最终解决方案，在任何其他通过软件解锁的方法失效情况下使用，但不建议过度使用，应当确认好程序配置再烧录，避免开发板损坏。

5.3 Keil 无法烧录 CM4 程序解决方案

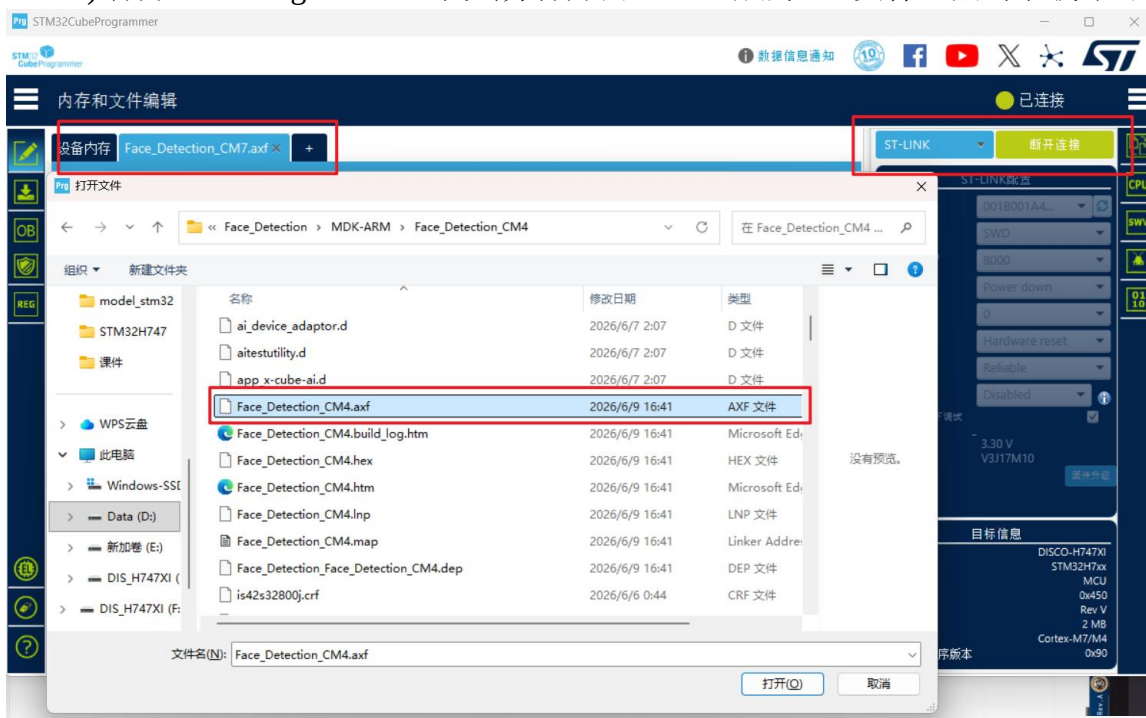
1) 使用 Keil 烧录完成 CM7 的程序之后，再烧录 CM4 程序，大概率出现以下错误。



2) 按照实验指导书的解决方案，可以修改 CM4 程序的 RAM 地址，即可解决这个问题。

3) 但是在部分情况下，即使修改 RAM 地址也无法解决这个问题，猜测与 Keil 本身存在的 bug 有关。考虑使用 CubeProgrammer 解决。

4) 打开 CubeProgrammer，找到并打开由 Keil 生成的 axf 文件，点击下载即可。



5) 其他方法：如果不在 CM4 中编程，可以先烧录 CM4 的程序，再烧录 CM7 的程序，按下 reset 按键启动或断电再插电重启；此后再修改 CM7 的代码，无需调整 CM4 的设置，直接烧录 CM7 的程序即可。

6) 奇怪表现：部分情况下，可能存在编译、下载正常但开发板没有反应（LCD 不亮、LED0 不亮），按下 reset 按键启动或断电再插电重启也没有效果；但此时选取之前的项目，如 GPIO_BSP 下载程序，开发板正常启动，再下载本实验的程序，则开发板 LCD 和 LED0 能正常点亮，但暂未发现具体原因。

6 实验收获与心得

本次实验完成了从硬件驱动到 AI 算法部署的完整嵌入式开发闭环，成功将 YuNet 人脸检测模型迁移至 STM32H747I-DISCO 开发板，打通了 ONNX 到嵌入式 C 代码的全流程，深刻体会了 AI 在资源受限硬件上的落地过程。

硬件层面，我们深入理解了 STM32H7 双核协同机制，掌握跨核配置与烧录方法。在 LCD 显示驱动中，认识到 DSI、LTDC、DMA2D 与 SDRAM 协同工作的复杂性，帧缓冲布局、显存带宽与 Cache 一致性对系统稳定性的决定性影响，强化了

底层硬件设计的系统观。

AI 部署方面，通过 STM32Cube AI Studio 工具链，攻克了模型量化压缩、张量格式对齐（BGR/RGB、NCHW）等工程壁垒。手工实现检测框解码、置信度筛选、NMS 及小框过滤等后处理算法，大幅提升了 C 语言重构与底层数学解析能力。

调试过程中，面对配置冲突、路径错位、串口丢包、画面撕裂及坐标偏移等问题，我们坚持“自底向上验证、模块化排查”的工程思维，深刻体会细微内存或时序偏差的严重后果，锤炼了严谨的调试习惯。

本次实验重塑了我们对边缘计算的认知，实现从 PC 端仿真到真实硬件落地的思维跃迁。嵌入式 AI 不仅要求算法有效，更对实时性、内存和长期稳定性提出严苛挑战。综合运用硬件配置、C 语言编程、Python 工具链与 AI 模型部署技术，极大丰富了工程实战经验，为后续智能终端与边缘计算研究奠定坚实技术基础。

参考资料

- [1] 意法半导体 (STMicroelectronics). 评估报告与指标: ST 边缘 AI 核心技术 4.0.0 版本 r2.7 [EB/OL]. 2026.
- [2] STMicroelectronics. Evaluation report and metrics[EB/OL]. [2026-06-13]. https://stedgeai-dc.st.com/assets/embedded-docs/evaluation_metrics.html#ref_valio_arg.
- [3] 意法半导体 (STMicroelectronics). STM32H747XI 探索套件用户手册: UM2411/Discovery kit with STM32H747XI MCU user manual (UM2411) [R]. 第 7 版。日内瓦: 意法半导体, 2025.
- [4] 意法半导体 (STMicroelectronics). 带 MIPI DSI 接口的 4 英寸 WVGA TFT LCD 电容触摸屏用户手册: UM2104/4-inch WVGA TFT LCD board with MIPI DSI interface and capacitive touch screen user manual (UM2104) [R]. 第 1 版。日内瓦: 意法半导体, 2016.
- [5] 意法半导体 (STMicroelectronics). 人工智能 (AI) X-CUBE-AI 扩展包入门指南/Artificial intelligence (AI) X-CUBE-AI expansion pack user manual: UM2526 [R]. 第 1 版。日内瓦: 意法半导体, 2021.
- [6] 意法半导体 (STMicroelectronics). STM32H747I-DISCO 开发板电路原理图: MB1248-D03/STM32H747I-DISCO development board schematic, board reference MB1248 [R]. D03 版。日内瓦: 意法半导体, 2022.
- [7] 嵌入式系统实验指导书(STM32H747I_DISCO_CUBEMX_HAL)